

INT 221 - MVC PROGRAMMING

Unit 5

CSRF Field

Cross-Site Request Forgery protection.

Prevents unauthorized form submissions.

Laravel Implementation

@csrf: -> Generates hidden input:

```
<input type="hidden" name="_token" value="...">
```

Method Field

HTML forms support only:

GET, POST

Laravel allows:

PUT, PATCH, DELETE

Syntax

```
@method('PUT')
```

Laravel Form Validation

Checking user input before processing.

Example

```
$request->validate([  
    'name' => 'required',  
]);
```

Validation Rules

Rule	Meaning
required	Field must be filled
email	Valid email
min:3	Minimum Length
max:10	Maximum Length
numeric	Must be number

Error Messages

Default

`$errors->all()`

Custom Messages

```
$request->validate([  
    'name' => 'required'  
], [  
    'name.required' => 'Name is mandatory.'  
]);
```

Custom Validation Rules

When default rules are not enough

Example: Name must start with capital letter

Repopulating Forms (Old Input)

```
value="{{ old('name') }}"
```

Keeps user input after validation error

MINI PROJECT

Student Registration System (Advanced Form Handling), Includes:

CSRF, Method Field, Validation + Rules, Custom Rule, Error Messages, Repopulating Form and Flash Message

PROJECT FLOW (IMPORTANT FOR EXAM)

Form → Submit → Validation

→ Error → Back with old input

→ Success → Store + Show Result

STEP 1: Create Laravel Project

```
composer create-project laravel/laravel student-app
```

```
cd student-app
```

```
php artisan serve
```

Open: <http://127.0.0.1:8000>



STEP 2: Create Controller-> php artisan make:controller StudentController

STEP 3: Create Custom Validation Rule-> php artisan make:rule CapitalName

app/Rules/CapitalName.php

```
<?php
namespace App\Rules;
use Closure;
use Illuminate\Contracts\Validation\ValidationRule;

class CapitalName implements ValidationRule{
    public function validate(string $attribute, mixed $value, Closure $fail): void{
        if (!ctype_upper(substr($value, 0, 1))) {
            $fail('The :attribute must start with a capital letter.');
```

STEP 4: Define Routes

`routes/web.php`

```
use Illuminate\Support\Facades\Route;  
use App\Http\Controllers\StudentController;
```

// Show form

```
Route::get('/student', [StudentController::class, 'form'])->name('student.form');
```

// Submit form

```
Route::post('/student', [StudentController::class, 'submit'])->name('student.submit');
```

// Method field demo (PUT)

```
Route::put('/student/update', [StudentController::class, 'update'])->name('student.update');
```

STEP 5: Controller Code

[app/Http/Controllers/StudentController.php](#)

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Rules\CapitalName;
class StudentController extends Controller{
    // Show Form
    public function form(){
        return view('student_form');
    }

    // Handle Form Submission
    public function submit(Request $request){
        // VALIDATION
        $validated = $request->validate([
            'name' => ['required', 'min:3', 'max:20', new CapitalName],
            'email' => 'required|email',
            'age' => 'required|numeric|min:18|max:60'
        ],
```



```
[
    'name.required' => 'Name is required!',
    'name.min' => 'Name must be at least 3 characters',
    'email.required' => 'Email is required!',
    'email.email' => 'Enter valid email!',
    'age.min' => 'Age must be at least 18',
    'age.max' => 'Age must be less than 60'
]);
// FLASH MESSAGE
session()->flash('success', 'Student Registered Successfully!');
// PASS DATA
return view('student_result', ['data' => $validated]);
}
// Method Field Demo
public function update()
{
    return "PUT method called successfully!";
}
}
```

STEP 6: Create two views

```
php artisan make:view student_form  
php artisan make:view student_result
```

STEP 7: Create Form View->resources/views/student_form.blade.php

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Student Form </title>  
    <!-- Bootstrap -->  
    <link  
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"rel="stylesheet">  
  </head>  
  <body class="container mt-5">  
    <h2>Student Registration </h2>  
    <!-- SUCCESS MESSAGE -->  
    @if(session('success'))  
      <div class="alert alert-success">  
        {{ session('success') }}  
      </div>  
    @endif
```

```
<form action="{{ route('student.submit') }}" method="POST">
```

```
<!-- CSRF -->
```

```
@csrf
```

```
<!-- NAME -->
```

```
<div class="mb-3">
```

```
<label>Name:</label>
```

```
<input type="text" name="name" class="form-control" value="{{ old('name') }}">
```

```
@error('name')
```

```
<small class="text-danger">{{ $message }}</small>
```

```
@enderror
```

```
</div>
```

```
<!-- EMAIL -->
```

```
<div class="mb-3">
```

```
<label>Email:</label>
```

```
<input type="email" name="email" class="form-control" value="{{ old('email') }}">
```

```
@error('email')
```

```
<small class="text-danger">{{ $message }}</small>
```

```
@enderror
```

```
</div>
```

```
<!-- AGE -->
<div class="mb-3">
  <label>Age:</label>
  <input type="number" name="age" class="form-control" value="{{ old('age') }}">
  @error('age')
    <small class="text-danger">{{ $message }}</small>
  @enderror
</div>
<button class="btn btn-primary">Submit</button>
</form><hr>
<!-- METHOD FIELD DEMO -->
<h4>PUT Method Demo</h4>
<form action="{{ route('student.update') }}" method="POST">
  @csrf
  @method('PUT')
  <button class="btn btn-warning">Call PUT Route</button>
</form>
</body>
</html>
```

STEP 8: Create Result View-> resources/views/student_result.blade.php

```
<!DOCTYPE html>
<html>
  <head>
    <title>Result</title>
  </head>
  <body>
    <h2>Student Details</h2>
    Name: {{ $data['name'] }} <br>
    Email: {{ $data['email'] }} <br>
    Age: {{ $data['age'] }} <br><br>
    <a href="{{ route('student.form') }}">Back</a>
  </body>
</html>
```

STEP 9: Run Project

php artisan serve

Open: <http://127.0.0.1:8000/student>

TEST CASES

Case 1: Empty Form-> Errors appear + old values retained

Case 2: Name not capital-> rabaab

Error: -> The name must start with a capital letter.

Case 3: Correct Data

Name: Rabaab

Email: test@gmail.com

Age: 22

Output: -> Student Registered Successfully!

The next page shows data

Case 4: PUT Button

Click button

Output:->PUT method called successfully!